
rxnet — Guía de usuario

FSM y Redes de Petri sobre un runtime síncrono reactivo

2026-04-16

Contents

rxnet — Guía de usuario	5
1. El problema que resuelve rxnet	5
2. La hipótesis síncrona	5
3. El tick y sus fases	6
Por qué esa separación de fases	6
4. Máquinas de estado finito (FSM)	7
4.1 Cuándo usar una FSM	7
4.2 Anatomía de una máquina	7
4.3 Guards y acciones	8
4.4 Ejemplo completo: luz con auto-apagado	8
4.5 El patrón de señales de entrada en FSM	10
5. Redes de Petri (PN)	10
5.1 Cuándo usar una red de Petri	10
5.2 Conceptos fundamentales	11
5.3 Semántica greedy-sequential	11
5.4 Anatomía de una red	12
5.5 Lugares de señal (signal places)	12
5.6 Ejemplo completo: blink con tres velocidades	13
6. Modelos de ejecución concurrente	14
6.1 Ejecutivo cíclico (Cyclic Executive)	14
6.2 Multitarea cooperativa (Cooperative Multitasking)	15
6.3 Threads con prioridades fijas y desalojo	17
6.4 Resumen comparativo	20
7. Cuándo usar FSM, cuándo PN	21
Usa FSM cuando...	21
Usa PN cuando...	21
Mezclar ambos (patrón habitual)	22
8. Referencia rápida de la API	22
Core runtime	22
FSM	23

Petri Net	23
Firmas de callbacks	24
Lectura adicional	24

List of Figures

rxnet — Guía de usuario

1. El problema que resuelve rxnet

Los sistemas reactivos responden a eventos del entorno: pulsaciones de botón, lecturas de sensor, mensajes de red, expiración de temporizadores. El reto no es *qué hacer* ante un evento, sino *cuándo hacerlo* y *qué hay que ver* en ese momento.

Sin disciplina, el código reactivo acumula dos tipos de bugs difíciles de reproducir:

- **Race conditions de lectura:** el guard de una transición lee un bit de hardware a mitad de ciclo, cuando otro módulo ya lo consumió.
- **Efectos secundarios prematuros:** una acción ejecutada durante la evaluación modifica estado que otro módulo todavía no evaluó.

rxnet impone una disciplina que los elimina por construcción: **toda la información que entra en el sistema se lee exactamente una vez por ciclo, al principio, y todos los efectos secundarios se ejecutan exactamente una vez, al final.**

2. La hipótesis síncrona

rxnet está inspirada en los *lenguajes síncronos* (Esterel, Lustre, Signal) y en sus derivados industriales (SCADE, Simulink). La idea central se llama la **hipótesis síncrona**:

La computación de un tick es instantánea. El mundo externo no cambia mientras el sistema está procesando.

Esto es una abstracción, no una afirmación física. En la práctica significa:

- El tick debe completarse antes de que llegue el siguiente evento significativo.
- Si el tick tarda más que el período de muestreo, el sistema tiene un problema de diseño, no de concurrencia.

Bajo la hipótesis síncrona, **no hay carrera de datos posible dentro del tick**: todos los módulos leen el mismo snapshot de entradas, y nadie ve los cambios de los demás hasta el siguiente tick.

3. El tick y sus fases

Cada llamada a `runtime.tick()` ejecuta seis fases en orden estricto:

#	Fase	Qué hace cada nodo
1	Latch inputs	<code>latch_inputs_cb()</code> — leer GPIO, calcular señales derivadas, tomar snapshot
2	Evaluate	<code>evaluate()</code> — decidir siguiente estado / flags (solo lectura del snapshot)
3	Commit	<code>commit()</code> — aplicar decisiones, encolar acciones diferidas
4	Deferred	ejecutar cola de acciones — timers, side-effects, notificaciones
5	Dump outputs	<code>dump_outputs_cb()</code> — escribir GPIO, imprimir estado

Por qué esa separación de fases

La separación **evaluate / commit / deferred** no es arbitraria:

- **Evaluate** solo puede leer, nunca escribir estado observable. Todos los módulos ven el mismo estado del sistema al mismo tiempo.
- **Commit** aplica las decisiones. Tras commit, el estado es consistente pero los efectos externos todavía no han ocurrido.

- **Deferred** ejecuta las acciones (arrancar un timer, encender un LED) *después* de que todo el mundo haya aplicado sus decisiones. Una acción ve el estado comprometido de todos los módulos, no solo el propio.

La separación **latch / dump** garantiza que las lecturas de hardware ocurren *antes* de evaluar (snapshot limpio) y las escrituras *después* de decidir (salida consistente).

4. Máquinas de estado finito (FSM)

4.1 Cuándo usar una FSM

Una FSM es la herramienta adecuada cuando el comportamiento del módulo depende de su **historia**: no solo del evento actual sino de lo que pasó antes.

Ejemplos típicos:

Problema	Estados naturales
Luz con botón	OFF, ON
Semáforo	ROJO, VERDE, AMARILLO
Puerta con cerrojo	ABIERTA, CERRADA, BLOQUEADA
Protocolo de comunicación	IDLE, CONECTANDO, CONECTADO, ERROR
Menú de display	MENU_PRINCIPAL, SUBMENU_A, SUBMENU_B

4.2 Anatomía de una máquina

```

1 from rxnet.fsm import Machine, Transition, Runtime
2
3 # Estados: constantes enteras (el runtime no sabe qué significan)
4 IDLE      = 0
5 RUNNING   = 1
6 ERROR     = 2
7
8 # Transiciones: (estado_origen, estado_destino, guardia?, acción?)
9 transitions = [
10     Transition(IDLE,    RUNNING, guard=button_pressed, action=
        start_motor),

```



```

11     Transition(RUNNING, IDLE, guard=button_pressed, action=
12         stop_motor),
13     Transition(RUNNING, ERROR, guard=fault_detected, action=
14         report_fault),
15     Transition(ERROR, IDLE, guard=reset_pressed, action=
16         clear_fault),
17 ]
18
19 machine = Machine(
20     name="motor",
21     state=IDLE,
22     transitions=transitions,
23     user=my_data, # datos de usuario, pasados a guard/
24     action
25     latch_inputs_cb=read_gpio, # llamada en fase 2
26     dump_outputs_cb=write_gpio, # llamada en fase 6
27 )
28
29 rt = Runtime()
30 rt.add_machine(machine)

```

Semántica first-match: en cada tick, el runtime recorre las transiciones en orden de declaración y ejecuta la *primera* cuya guardia sea verdadera y cuyo estado origen coincida con el estado actual. Si ninguna coincide, la máquina permanece en su estado.

4.3 Guards y acciones

```

1 from rxnet.runtime import Context
2
3 # Guard: función pura que devuelve bool
4 # Recibe el contexto del tick y los datos de usuario
5 def button_pressed(ctx: Context, data: MyData) -> bool:
6     return data.latched_button_event # leer snapshot, nunca hardware
7     en vivo
8
9 # Acción: función deferred (corre en fase 5, después de commit)
10 # En este punto todos los módulos ya aplicaron sus cambios
11 def start_motor(ctx: Context, data: MyData) -> None:
12     data.motor_enabled = True
13     # Aquí podemos consultar otros módulos: su estado ya está
14     comprometido

```

Regla importante: los guards nunca deben tener efectos secundarios. Son funciones puras de lectura. Los efectos van en las acciones.

4.4 Ejemplo completo: luz con auto-apagado

```
1 import time
2 from rxnet.fsm import Machine, Transition, Runtime
3 from rxnet.runtime import Context
4
5 # -- Estados --
6 OFF = 0
7 ON  = 1
8
9 # -- Datos de la máquina --
10 class LightData:
11     def __init__(self, timeout_ms: int):
12         self.latched_event = False
13         self.enabled = False
14         self.timeout_ms = timeout_ms
15         self.deadline_ms = 0
16         self.now_ms = 0
17
18     def now_ms() -> int:
19         return int(time.monotonic() * 1000)
20
21 # -- Callbacks de fase --
22 def latch(ctx: Context, d: LightData) -> None:
23     d.now_ms = now_ms()
24     d.latched_event = read_button() # snapshot del hardware
25
26 def dump(ctx: Context, d: LightData) -> None:
27     set_led(d.enabled)
28
29 # -- Guards --
30 def pressed(ctx: Context, d: LightData) -> bool:
31     return d.latched_event
32
33 def timed_out(ctx: Context, d: LightData) -> bool:
34     return d.now_ms >= d.deadline_ms
35
36 # -- Acciones (deferred: ven el estado comprometido) --
37 def turn_on(ctx: Context, d: LightData) -> None:
38     d.enabled = True
39     d.deadline_ms = now_ms() + d.timeout_ms
40
41 def turn_off(ctx: Context, d: LightData) -> None:
42     d.enabled = False
43
44 # -- Montaje --
45 data = LightData(timeout_ms=5000)
46 machine = Machine(
47     name="light",
48     state=OFF,
49     transitions=[
50         Transition(OFF, ON, guard=pressed, action=turn_on),
```

```

51     Transition(ON, ON, guard=pressted, action=turn_on), # reset
        timer
52     Transition(ON, OFF, guard=timed_out, action=turn_off),
53 ],
54 user=data,
55 latch_inputs_cb=latch,
56 dump_outputs_cb=dump,
57 )
58
59 rt = Runtime()
60 rt.add_machine(machine)
61
62 # Bucle principal (10 ms)
63 while True:
64     rt.tick()
65     time.sleep(0.010)

```

4.5 El patrón de señales de entrada en FSM

La forma canónica de conectar hardware a una FSM en rxnet:

Hardware	<code>latch_inputs_cb</code>	guards / actions
GPIO → evento (vivo, volátil)	<code>data.latched = True</code> (snapshot estable)	<code>if data.latched: ...</code> (solo leen snapshot)

El callback `latch_inputs_cb` es el único punto de contacto con el hardware. El resto de la máquina trabaja con copias estables.

5. Redes de Petri (PN)

5.1 Cuándo usar una red de Petri

Una red de Petri es la herramienta adecuada cuando hay **conurrencia natural**: varios procesos que avanzan en paralelo y necesitan sincronizarse.

Ejemplos típicos:

Problema	Modela bien porque...
Botón toggle	Un token fluye entre P_OFF y P_ON
Buffer productor-consumidor	Tokens representan slots llenos/vacíos
Semáforo / mutex	Un token en P_LIBRE controla el acceso
Protocolo de handshake	Ambos lados avanzan y se sincronizan
Recursos compartidos	Tokens cuentan instancias disponibles

La PN no reemplaza a la FSM: cuando el comportamiento es esencialmente lineal (un agente con historia), la FSM es más clara. Cuando hay múltiples flujos que se sincronizan, la PN es más clara.

5.2 Conceptos fundamentales

Concepto	Símbolo	Significado
Lugar (place)	○	condición, estado, recurso, mensaje pendiente
Token	•	unidad de recurso, condición activa
Transición	rect.	evento, paso de proceso
Arco de entrada	→T	condición necesaria para disparar (consume tokens)
Arco de salida	T→	condición que produce al disparar (produce tokens)

Una transición **está habilitada** cuando todos sus lugares de entrada tienen suficientes tokens ($\geq$ peso del arco). Una transición habilitada **dispara**: consume los tokens de sus entradas y produce tokens en sus salidas.

5.3 Semántica greedy-sequential

rxnet usa evaluación **greedy-sequential**: las transiciones se evalúan en orden de declaración, y las anteriores *consumen tokens inmediatamente*, antes de que las posteriores se evalúen.

Consecuencia práctica: **el orden de declaración implica prioridad**. Si dos transiciones compiten por el mismo token, gana la que se declara primero.

```
1 transitions = [
2     # T0 declarada antes → prioridad sobre T1
3     Transition(consume=[Arc(P_X, 1)], produce=[Arc(P_A, 1)]), # T0
4     Transition(consume=[Arc(P_X, 1)], produce=[Arc(P_B, 1)]), # T1 (no
5     # dispara si T0 disparó)
6 ]
```

5.4 Anatomía de una red

```
1 from rxnet.pn import Net, Transition, Arc, Runtime
2
3 # Lugares: índices 0, 1, 2, ...
4 P_OFF      = 0
5 P_ON       = 1
6 P_REQUEST  = 2
7
8 net = Net(
9     name="light",
10    places=[1, 0, 0], # token inicial en P_OFF
11    transitions=[
12        # off + request → on
13        Transition(consume=[Arc(P_OFF, 1), Arc(P_REQUEST, 1)],
14                  produce=[Arc(P_ON, 1)]),
15        # on + request → off
16        Transition(consume=[Arc(P_ON, 1), Arc(P_REQUEST, 1)],
17                  produce=[Arc(P_OFF, 1)]),
18    ],
19 )
20
21 rt = Runtime()
22 rt.add_net(net)
```

5.5 Lugares de señal (signal places)

Un **lugar de señal** no acumula tokens: se *restablece* en cada latch a 0 o 1 según una condición del entorno. Es el equivalente PN de los inputs latched de una FSM.

```
1 # En latch_inputs_cb: reescribir P_TOGGLE_DUE en cada tick
2 def latch_cb(ctx, _user):
3     is_blinking = net.places[P_X1] > 0 or net.places[P_X2] > 0
4     net.places[P_TOGGLE_DUE] = 1 if (is_blinking and timer_elapsed())
5     else 0
```

Contrasta con los **lugares de cola** (como P_REQUEST), que acumulan tokens:

```
1 # En latch_inputs_cb: añadir un token si hay pulsación
2 def latch_cb(ctx, _user):
3     if button_pressed():
4         net.places[P_REQUEST] += 1    # acumula; se consume 1 por tick
```

Tipo de lugar	Comportamiento	Ejemplo
Señal	Se reescribe a 0/1 cada latch	Timer expirado, sensor activo
Cola	Acumula; transición consume 1	Pulsación de botón pendiente
Estado	Token único que fluye	ON/OFF, ROJO/VERDE
Contador	N tokens = N recursos	Slots libres en buffer

5.6 Ejemplo completo: blink con tres velocidades

El botón avanza cíclicamente entre modos: OFF → X1 → X2 → OFF. Dentro de X1 y X2 hay un self-loop de toggle que invierte el LED cada semiperíodo (a velocidad base en X1, al doble en X2).

```
1 P_OFF, P_X1, P_X2, P_REQUEST, P_TOGGLE_DUE = 0, 1, 2, 3, 4
2
3 def create_blink_pn(button_gpio, light_gpio, base_hz):
4     state = {"output": False, "next_toggle_ms": 0, "now_ms": 0}
5
6     def action_enter_x1(ctx, _): state["output"] = True; ...
7     def action_enter_x2(ctx, _): state["output"] = True; ...
8     def action_enter_off(ctx, _): state["output"] = False; state["next_toggle_ms"] = 0
9     def action_toggle(ctx, _): state["output"] = not state["output"]; ...
10
11     net = Net(
12         name="blink",
13         places=[1, 0, 0, 0, 0],
14         transitions=[
15             # Transiciones de botón ANTES que las de toggle (prioridad greedy)
16             Transition(consume=[Arc(P_OFF,1), Arc(P_REQUEST,1)],
17                       produce=[Arc(P_X1,1)], action=action_enter_x1),
17             Transition(consume=[Arc(P_X1,1), Arc(P_REQUEST,1)], produce=[Arc(P_X2,1)],
18                       action=action_enter_x2),
18             Transition(consume=[Arc(P_X2,1), Arc(P_REQUEST,1)], produce=[Arc(P_OFF,1)],
19                       action=action_enter_off),
19             # Self-loops de toggle DESPUÉS (pierden si hay botón en el mismo tick)
```

```

20         Transition(consume=[Arc(P_X1,1), Arc(P_TOGGLE_DUE,1)],
21                   produce=[Arc(P_X1,1)], action=action_toggle),
22         Transition(consume=[Arc(P_X2,1), Arc(P_TOGGLE_DUE,1)],
23                   produce=[Arc(P_X2,1)], action=action_toggle),
24     ],
25     # ... latch/dump callbacks
26     return net

```

6. Modelos de ejecución concurrente

Esta sección explica cómo rxnet se relaciona con los tres modelos clásicos de ejecución en sistemas embebidos y de tiempo real.

6.1 Ejecutivo cíclico (Cyclic Executive)

Qué es

Un ejecutivo cíclico es el modelo más simple de concurrencia: un único hilo de ejecución que repite la misma secuencia de tareas indefinidamente, con un período fijo.

Cada período (p. ej. 10 ms) ejecuta en orden fijo las tareas A (sensores), B (control) y C (actuadores), sin interrupciones entre ellas.

rxnet ES un ejecutivo cíclico

El bucle principal de cualquier ejemplo rxnet es un ejecutivo cíclico:

```

1  # Esto es literalmente un ejecutivo cíclico
2  PERIOD_S = 0.010    # 10 ms
3
4  next_tick = time.monotonic()
5  while True:
6      rt.tick()        # ← el "frame" del ejecutivo
7
8      next_tick += PERIOD_S
9      sleep_s = next_tick - time.monotonic()
10     if sleep_s > 0:
11         time.sleep(sleep_s)

```

Cada nodo registrado en el runtime es una “tarea” del ejecutivo. El orden de registro determina el orden de ejecución dentro del frame:

```
1 rt.add_machine(sensor_machine)    # tarea 1: se evalúa primero
2 rt.add_machine(control_machine)   # tarea 2: ve el estado comprometido
   de tarea 1
3 rt.add_machine(actuator_machine)  # tarea 3: ve el estado de las
   anteriores
```

Ventajas del ejecutivo cíclico: - Determinista: mismo orden en cada ciclo - Sin overhead de scheduler - Sin posibilidad de deadlock o starvation - Fácil de analizar temporalmente: si cada tarea tarda $\leq T$, el ciclo tarda $\leq N \times T$

Limitaciones: - Todas las tareas comparten el mismo período - Una tarea lenta retrasa todas las demás - No hay forma de responder a eventos urgentes a mitad de ciclo

Ajuste fino: múltiples runtimes a distintas velocidades

Para tareas con períodos diferentes, se pueden usar múltiples runtimes con bucles independientes, o ejecutar algunos nodos con división de frecuencia:

```
1 tick_count = 0
2 while True:
3     fast_runtime.tick()           # siempre: 10 ms
4     if tick_count % 10 == 0:
5         slow_runtime.tick()      # cada 100 ms
6     tick_count += 1
7     time.sleep(0.010)
```

6.2 Multitarea cooperativa (Cooperative Multitasking)

Qué es

En la multitarea cooperativa, cada tarea cede el control voluntariamente cuando termina su trabajo. No hay interrupciones; el scheduler solo actúa en los puntos de *yield*. Python [asyncio](#) y MicroPython son ejemplos modernos.

	Ejecutando	Esperando	Ejecutando	Esperando
Tarea A	bloque 1	cede (yield)	bloque 2	cede
Tarea B	—	bloque 1	—	bloque 2

rxnet como multitarea cooperativa

En rxnet, cada nodo *cede implícitamente* al terminar su `evaluate()`. El runtime es el scheduler; `tick()` es la ronda de scheduling.

La diferencia conceptual respecto al ejecutivo cíclico es que una “tarea” puede necesitar **múltiples ticks** para completar su trabajo. El estado de la tarea (en qué punto del trabajo está) se modela explícitamente como estado de FSM o como distribución de tokens en la PN.

Ejemplo: tarea que procesa una cola de mensajes, uno por tick

La máquina arranca en `IDLE`. Al detectar un mensaje transiciona a `PROCESANDO` (un tick = un paso). Si el proceso termina bien vuelve a `IDLE`; si hay error va a `ERROR_HANDLER`.

```
1  IDLE          = 0
2  PROCESANDO    = 1
3  ERROR_HANDLER = 2
4
5  class WorkerData:
6      def __init__(self, queue):
7          self.queue = queue
8          self.current_msg = None
9
10 def latch(ctx, d: WorkerData):
11     # Tomamos UN mensaje por tick (no vaciamos toda la cola)
12     d.current_msg = d.queue.pop(0) if d.queue else None
13
14 def has_message(ctx, d): return d.current_msg is not None
15 def is_processed(ctx, d): return True # en este tick simple, siempre
    termina
16 def has_error(ctx, d): return d.current_msg and d.current_msg.
    is_invalid()
17
18 def process(ctx, d):
19     do_work(d.current_msg) # acción deferred: todos los módulos ya
    commitaron
20
21 machine = Machine(
22     name="worker",
23     state=IDLE,
24     transitions=[
25         Transition(IDLE, PROCESANDO, guard=has_message),
26         Transition(PROCESANDO, IDLE, guard=is_processed,
27             action=process),
28         Transition(PROCESANDO, ERROR_HANDLER, guard=has_error),
29     ],
30     user=WorkerData(my_queue),
31     latch_inputs_cb=latch,
32 )
```

Comunicación entre tareas cooperativas

Las tareas se comunican a través de:

1. Lugares de PN (tokens como mensajes):

```
1 # El productor añade un token en P_BUFFER cuando tiene datos
2 # El consumidor consume ese token cuando está listo
3 P_BUFFER = 2
4 net.places[P_BUFFER] += 1 # en latch_inputs_cb del productor
```

2. Estado de FSM como señal de coordinación:

```
1 # Tarea B puede leer el estado de tarea A en sus guards
2 def a_is_ready(ctx, d):
3     return task_a_machine.state == READY # acceso directo al estado
```

3. Context inputs (datos globales del tick):

```
1 # En el context global se puede poner un struct de señales
2 # compartidas que todas las máquinas leen durante latch
3 ctx.latched_inputs.alarm_active = sensor_data.alarm
```

Diferencia clave con el ejecutivo cíclico

	Ejecutivo cíclico	Multitarea cooperativa
Unidad de trabajo	Todo en un tick	Puede abarcar N ticks
Estado entre ticks	No necesario	Capturado en estados/tokens
Comunicación	Secuencia fija	Señales explícitas
Modelo rxnet	Nodos sin estado persistente	FSM / PN con estado

6.3 Threads con prioridades fijas y desalojo

Qué es

En un RTOS (FreeRTOS, Zephyr, ThreadX), las tareas tienen prioridades numéricas. El scheduler **desaloja** (preempt) una tarea de baja prioridad cuando una de alta prioridad pasa a estar lista para ejecutar, incluso si la primera no ha terminado su trabajo.

	t1	t2	t3	t4
Alta prioridad	—	ejecuta	—	ejecuta
Baja prioridad	ejecuta	<i>preemptada</i>	ejecuta	<i>preemptada</i>

¿Puede rxnet modelar esto?

Directamente, **no**: rxnet es síncrono y cooperativo. Un tick es atómico; nada puede interrumpirlo a mitad.

Pero rxnet puede usarse de dos formas en un sistema con hilos preemptivos:

Forma 1: rxnet dentro de una tarea RTOS (lo más común)

rxnet corre dentro de una única tarea RTOS. El RTOS gestiona las prioridades reales entre tareas del sistema; rxnet gestiona la lógica de la aplicación dentro de su tarea.

- RTOS scheduler
 - Tarea 1 — alta prioridad: IRQ de emergencia
 - Tarea 2 — media prioridad: **rxnet corre aquí** (`rt.tick()` cada 10 ms)
 - * `sensor_machine`
 - * `control_machine`
 - * `actuator_machine`
 - Tarea 3 — baja prioridad: logging, comms

El RTOS garantiza que la tarea rxnet se ejecuta con su prioridad. Dentro de esa tarea, el ejecutivo cíclico de rxnet gestiona las sub-tareas sin overhead de RTOS.

Forma 2: modelar prioridades dentro de rxnet

Si *todas* las “tareas” son manejadas por rxnet (sin RTOS), se puede modelar comportamiento de prioridades usando las propiedades del modelo:

a) Prioridad por orden de nodo

Los nodos registrados antes se evalúan primero. En PN con semántica greedy-sequential, las transiciones declaradas antes tienen prioridad sobre las declaradas después para el mismo recurso:

```

1 transitions = [
2     # Prioridad alta: esta transición se evalúa primero
3     # Si dispara, consume el token y la de baja prioridad no puede
4     Transition(consume=[Arc(P_CPU, 1)], produce=[Arc(P_TASK_HIGH, 1)],
5               guard=high_prio_ready),      # ← alta prioridad
6
7     # Prioridad baja: solo dispara si la alta no consumió el token
8     Transition(consume=[Arc(P_CPU, 1)], produce=[Arc(P_TASK_LOW, 1)],
9               guard=low_prio_ready),      # ← baja prioridad
10 ]

```

b) Modelar desalojo con señales

El “desalojo” se puede modelar como una transición que devuelve el token de CPU al pool cuando llega una tarea de mayor prioridad:

Estado normal: `P_CPU=1`, `P_LOW_RUNNING=1`

Llega alta prioridad: `T_PREEMPT`: consume `{P_LOW_RUNNING, P_HIGH_READY}`, produce `{P_HIGH_RUNNING, P_LOW_SUSPENDED}`

Alta prioridad termina: `T_RESUME`: consume `{P_HIGH_DONE, P_LOW_SUSPENDED}`, produce `{P_LOW_RUNNING}`

```

1 P_CPU          = 0    # token = CPU libre
2 P_HIGH_READY   = 1
3 P_HIGH_RUNNING = 2
4 P_LOW_READY    = 3
5 P_LOW_RUNNING  = 4
6 P_LOW_SUSPENDED = 5
7
8 transitions = [
9     # Despacho normal de alta prioridad
10    Transition(consume=[Arc(P_CPU,1), Arc(P_HIGH_READY,1)],
11              produce=[Arc(P_HIGH_RUNNING,1)]),
12
13    # Despacho normal de baja prioridad (solo si no hay alta lista)
14    Transition(consume=[Arc(P_CPU,1), Arc(P_LOW_READY,1)],
15              produce=[Arc(P_LOW_RUNNING,1)]),
16
17    # Preempción: alta prioridad interrumpe a baja
18    Transition(consume=[Arc(P_LOW_RUNNING,1), Arc(P_HIGH_READY,1)],
19              produce=[Arc(P_LOW_SUSPENDED,1), Arc(P_HIGH_RUNNING,1)])
20    ,
21    # Reanudación: alta termina, baja retoma CPU
22    Transition(consume=[Arc(P_HIGH_RUNNING,1), Arc(P_LOW_SUSPENDED,1)],
23              produce=[Arc(P_CPU,1), Arc(P_LOW_RUNNING,1)]),
24 ]

```

Nota importante: este modelo describe *cuándo* debe ejecutarse cada tarea, pero rxnet no ejecuta código en paralelo real. Para ejecución paralela verdadera, se necesita combinar con hilos del SO o del RTOS. rxnet modela la **política** de scheduling; el SO implementa el **mecanismo**.

Forma 3: múltiples runtimes tick a distintas frecuencias

Para sistemas con tareas de períodos muy distintos:

```

1 # Tarea de alta frecuencia: control (1 ms)
2 fast_rt = Runtime()
3 fast_rt.add_machine(control_machine)
4
5 # Tarea de baja frecuencia: monitorización (100 ms)
6 slow_rt = Runtime()
7 slow_rt.add_machine(monитор_machine)
8
9 tick_count = 0
10 next_tick = time.monotonic()
11
12 while True:
13     fast_rt.tick()           # cada 1 ms
14     if tick_count % 100 == 0:
15         slow_rt.tick()      # cada 100 ms
16
17     tick_count += 1
18     next_tick += 0.001
19     time.sleep(max(0, next_tick - time.monotonic()))

```

Este patrón es equivalente a un ejecutivo cíclico con **trama mayor** (major frame) y **tramas menores** (minor frames), el diseño estándar de ejecutivos cíclicos en aviación (DO-178C) y automoción (AUTOSAR).

6.4 Resumen comparativo

Modelo	rxnet cómo lo implementa	Limitación
Ejecutivo cíclico	<code>rt.tick()</code> en bucle con <code>sleep</code>	Todas las tareas al mismo período

Modelo	rxnet cómo lo implementa	Limitación
Multitarea cooperativa	Estado en FSM/PN, comunicación por tokens o señales	Sin preempción real
Prioridad por policy	Orden de nodos + greedy-sequential PN	Solo modela la política, no el mecanismo
Hilos preemptivos	rxnet dentro de una tarea RTOS	El SO gestiona la preempción real
Multi-rate	Múltiples runtimes a distintas frecuencias	Mayor/minor frames manuales

7. Cuándo usar FSM, cuándo PN

La pregunta más frecuente: ¿qué modelo uso?

Usa FSM cuando...

- El módulo tiene **un agente con historia**: hay un único “estado del módulo” en cada momento.
- Las transiciones son mutuamente excluyentes: estar en un estado excluye estar en otro.
- El control de flujo es lineal (incluso si tiene bucles y ramas).
- Necesitas guards complejos que lean múltiples variables.
- Semáforo: ROJO → VERDE → AMARILLO → ROJO
- Cerrojo: DESBLOQUEADO → BLOQUEADO
- Protocolo: IDLE → SYN_SENT → ESTABLISHED

Usa PN cuando...

- Hay **múltiples agentes independientes** que se sincronizan.
- Los recursos son contables (N slots, M conexiones disponibles).

- El paralelismo es natural: varias actividades ocurren al mismo tiempo.
- Necesitas modelar producción/consumo de forma explícita.
- Productor/consumidor: places `P_LLENOS` + `P_VACIOS`
- Semáforo binario: `P_MUTEX` (0 o 1 token)
- Rendezvous: A espera a B, B espera a A

Mezclar ambos (patrón habitual)

Lo más potente es combinarlos. La FSM describe la **política** (qué modo estoy); la PN describe los **recursos y sincronización** (qué tengo disponible):

```

1 # Sistema de acceso a sala:
2 # FSM: gestiona el ciclo de apertura de puerta (CERRADA → ABRIENDO →
   ABIERTA → CERRANDO)
3 # PN: gestiona los recursos (tokens = personas dentro, capacidad máxima
   )
4
5 door_machine = Machine(...) # controla la puerta
6 capacity_net = Net(...)    # cuenta ocupantes
7
8 fsm_rt = Runtime()
9 fsm_rt.add_machine(door_machine)
10
11 pn_rt = Runtime()
12 pn_rt.add_net(capacity_net)
13
14 while True:
15     fsm_rt.tick()    # primero: el FSM decide si abrir
16     pn_rt.tick()    # después: la PN actualiza el conteo
17     time.sleep(0.010)

```

8. Referencia rápida de la API

Core runtime

```

1 from rxnet.runtime import Context, Runtime
2
3 rt = Runtime(inputs=my_inputs)    # inputs opcionales
4 rt.add_node(node)                # registrar nodo (FSM o PN o custom)

```

```

5  rt.tick()                # ejecutar un ciclo completo
6  rt.context                # acceder al contexto
7
8  ctx.inputs                # entradas en vivo (mutables)
9  ctx.latched_inputs        # snapshot de este tick (solo lectura)
10 ctx.enqueue_deferred_action(fn, user) # encolar acción para fase 5

```

FSM

```

1  from rxnet.fsm import Machine, Transition, Runtime
2
3  # Transición mínima (sin guard = siempre dispara)
4  Transition(from_state=A, to_state=B)
5
6  # Transición completa
7  Transition(from_state=A, to_state=B, guard=my_guard, action=my_action)
8
9  # Máquina mínima
10 Machine(name="m", state=INITIAL, transitions=[...])
11
12 # Máquina con callbacks de fase
13 Machine(
14     name="m",
15     state=INITIAL,
16     transitions=[...],
17     user=my_data,
18     latch_inputs_cb=read_inputs,    # fase 2: snapshot hardware
19     dump_outputs_cb=write_outputs,  # fase 6: escribir hardware
20 )
21
22 rt = Runtime()
23 rt.add_machine(machine)
24 rt.tick()

```

Petri Net

```

1  from rxnet.pn import Arc, Net, Transition, Runtime
2
3  # Arco
4  Arc(place_id=0, weight=1)    # consume/produce 1 token del lugar 0
5
6  # Transición
7  Transition(
8      consume=[Arc(P_A, 1), Arc(P_B, 2)], # necesita 1 de A y 2 de B
9      produce=[Arc(P_C, 1)],              # produce 1 en C
10     guard=my_guard,                      # opcional
11     action=my_action,                   # deferred, opcional

```



```
12 )
13
14 # Red
15 Net(
16     name="n",
17     places=[1, 0, 0],          # marcado inicial
18     transitions=[...],
19     user=my_data,
20     latch_inputs_cb=read_inputs,
21     dump_outputs_cb=write_outputs,
22 )
23
24 rt = Runtime()
25 rt.add_net(net)
26 rt.tick()
```

Firmas de callbacks

```
1 from rxnet.runtime import Context
2
3 # Guard: pura, sin efectos, devuelve bool
4 def my_guard(ctx: Context, user: MyData) -> bool:
5     return user.latched_event
6
7 # Acción: deferred, corre en fase 5 con estado ya comprometido
8 def my_action(ctx: Context, user: MyData) -> None:
9     user.output = True
10
11 # Latch: snapshot hardware, actualizar señales derivadas
12 def latch_cb(ctx: Context, user: MyData) -> None:
13     user.latched_event = read_gpio(user.button_pin)
14
15 # Dump: escribir salidas al hardware
16 def dump_cb(ctx: Context, user: MyData) -> None:
17     write_gpio(user.led_pin, user.output)
```

Lectura adicional

- **Código fuente:** [python/rxnet/](#) — runtime (~80 líneas), fsm (~80 líneas), pn (~130 líneas)
- **Tests:** [python/tests/](#) — 56 tests que documentan el comportamiento esperado
- **Ejemplos interactivos:** [python/examples/fsm/](#) y [python/examples/pn/](#)

- **Especificaciones:** [docs/specs/python/requirements.md](#) y [docs/specs/python/design.md](#)
- **Implementación C:** `c/` — misma semántica, API equivalente en C11